

Tutorial 2 - Create a Jigsaw Tile with Image

First, create a script folder and create a new C# script with the name 'TempleBezierCurve' and attach it to an empty.

Let's create a set of central points that will define our Bezier curve to cut out jigsaw tiles. Therefore, we found a set of control points below to fit the jigsaw curve.

```
using NUnit.Framework;
using System.Collections.Generic;
using UnityEngine;

[Unity Script | 1 reference]
public class TemplebeizerCurve : MonoBehaviour
{
    public static readonly List<Vector2> templateControlPoints = new List<Vector2>
    {
        new Vector2(0, 0),
        new Vector2(35, 15),
        new Vector2(47, 13),
        new Vector2(45, 5),
        new Vector2(48, 0),
        new Vector2(25, -5),
        new Vector2(15, -18),
        new Vector2(36, -20),
        new Vector2(64, -20),
        new Vector2(85, -18),
        new Vector2(75, -5),
        new Vector2(52, 0),
        new Vector2(55, 5),
        new Vector2(53, 13),
        new Vector2(65, 15),
        new Vector2(100, 0)
    };
};
```

These control points are for x from 0 to 100 and y from -20 to 20.

Afterwards, in the TempleBezierCurve copy and paste the code we created in Bezier in the first tutorial.

Replace the variable `public List<Vextor2> controlPoints` with `public static readonly List<Vector2>TempleControlPoints` and use the control points shown here.

```
private void Start()
{
    mLineRenderers = new LineRenderer[2];
    mLineRenderers[0] = CreateLine();
    mLineRenderers[1] = CreateLine();

    mLineRenderers[0].gameObject.name = "LineRenderer_obj_0";
    mLineRenderers[1].gameObject.name = "LineRenderer_obj_1";

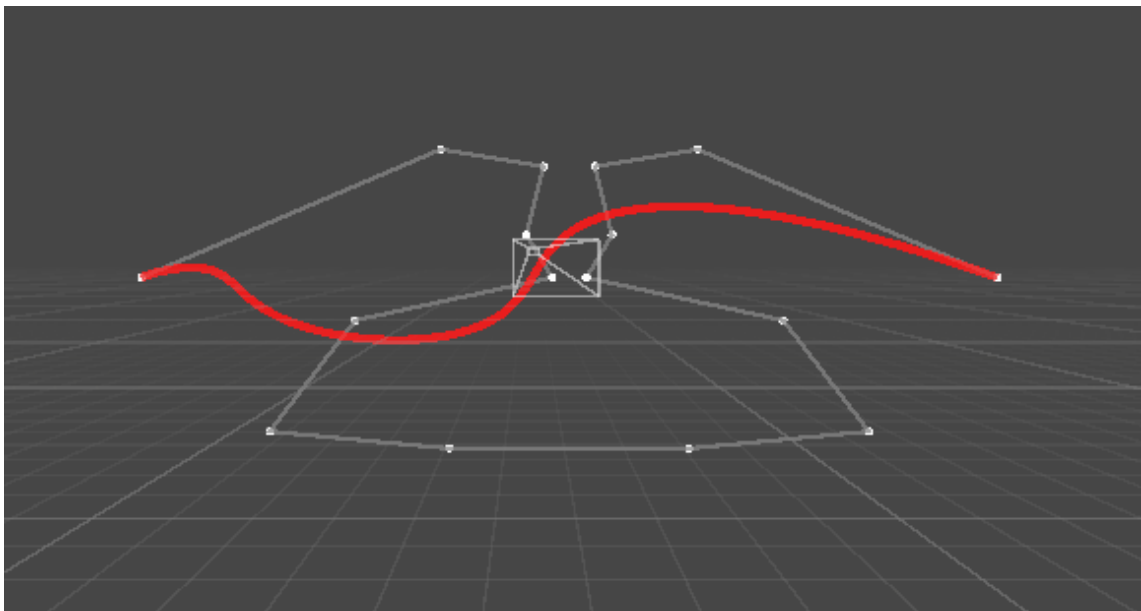
    for (int i = 0; i < templateControlPoints.Count; i++)
    {
        GameObject obj = Instantiate(PointPrefab, templateControlPoints[i], Quaternion.identity);
        obj.name = "ControlPoint_" + i.ToString();
        mPointGameObjects.Add(obj);
    }
}
```

NOTE: By making the TemplateControlPoints list static and read-only provides several advantages as it ensures that its content can not be modified after initialisation. This helps

Afterwards, go to the Unity editor and associate the Point prefab. Then, set the LineWidth and LineWidthBezier to 0.05 and 1, respectively.

Select the MainCamera game object and change the transform position x to 50 and size to 50 which it'll make the position the view better to see in the results.

Click Play and you should see the temple Bezier curve.



Sprite Utility Functions

Now, create some utility functions for sprites. Create a new C# file and called it SpriteUtils. In the script, we will implement these utility functions for sprites.

There are two static methods for working with sprites textures. The first one is called CreateSpriteFromTexture2D. This method will take a Texture2D object along with the parameters specifying a rectangular region within that texture. It will then create a Sprite object based on the region using the Sprite.Create method.

```
using UnityEngine;

2 references
public class SpriteUtils
{
    1 reference
    public static Sprite CreateSpriteFromTexture2D(
        Texture2D spriteTexture,
        int x,
        int y,
        int w,
        int h,
        float pixelPerUnit = 1.0f,
        SpriteMeshType spriteType = SpriteMeshType.Tight)
    {
        Sprite newSprite = Sprite.Create(
            spriteTexture,
            new Rect(x, y, w, h),
            new Vector2(0.5f, 0.5f),
            pixelPerUnit,
            0,
            spriteType);
        return newSprite;
    }
}
```

The PixelPerUnit parameter will determine how many pixels in the texture correspond to one unit in world space. The spriteType parameter will specify the type of mesh generated for the sprite.

```
public static Texture2D LoadTexture(string resourcePath)
{
    Texture2D tex = Resources.Load<Texture2D>(resourcePath);
    return tex;
}
}
```

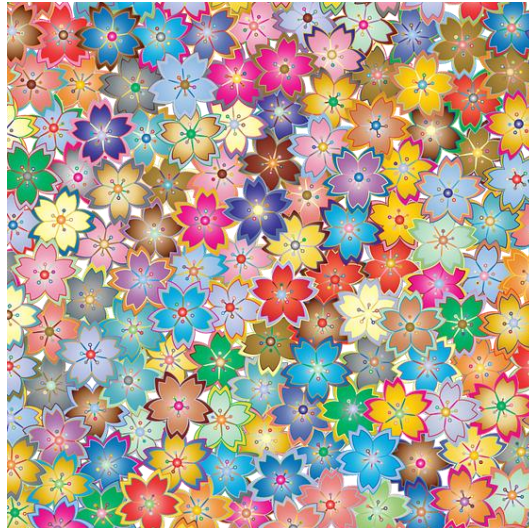
The second method allows the Texture2D object to load from a specified resource path using Unity's Resources.Load method. It takes a string resourcePath as an input, which represents the path to the texture resource within the project's Resources folder. The second method is called LoadTexture.

These two methods will provide convenient utilities for loading textures and creating sprites which can be helpful in dynamically generating or manipulating sprites within our application.

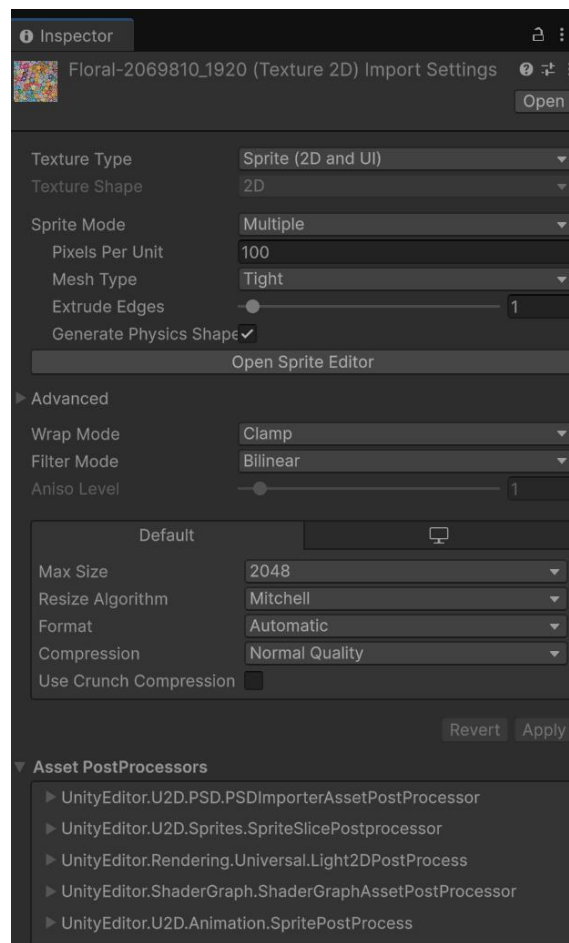
Image Tile

Next, we will load the image, where we will apply our template Bezier curve to create the jigsaw tiles.

Open the Unity editor, right- click on the Projects window and create a new folder called Images in Resources. Use a square image to use as an image tile.



We will use the image to create the jigsaw tile. Go to Unity editor, select the image you chose and go to the inspector, select Advanced and enable the Read/Write checkbox.



Add a public string variable named `imageFilename`. This variable will store the name of the image to which we want to apply the Bezier curve to create the jigsaw tile. Also, we'll be adding another private variable of type `Texture2D` of name `mTextureOriginal`. This means the variable will store the original texture loaded from the image file.

```
// Start is called once before the first execution of Update after the MonoBehaviour is created
public string imageFilename;
private Texture2D mTextureOriginal;
private Tile mTile;
```

Next, we create method called `CreateBaseTexture`. This method creates the base texture and displaying it as a sprite within the Unity scene. It loads the main image texture using the `SpriteUtile.LoadTexture` method, passing the `imageFilename`. It checks if the loaded texture is readable. If not, it'll log a message and returns, indicating that the texture cannot be used further as it adds a `SpriteRenderer` component to the current `GameObject` to display the sprite.

```

void CreateBaseTexture()
{
    mTextureOriginal = SpriteUtils.LoadTexture(imageFilename);
    if(!mTextureOriginal.isReadable)
    {
        Debug.Log("Texture load is unreadable ");
        return;
    }

    SpriteRenderer spriteRenderer = gameObject.GetComponent<SpriteRenderer>();
    // Replace the call to the non-existent CreateSpriteTexture2D with the correct method CreateSpriteFreeTexture2D
    spriteRenderer.sprite = SpriteUtils.CreateSpriteFreeTexture2D(
        mTextureOriginal,
        0,
        0,
        mTextureOriginal.width,
        mTextureOriginal.height);
}

```

This sets the sprite of the SpriteRenderer component using the SpriteUtils.CreateSpriteFromTexture2D method, passing in the loaded texture and. Go to the Unity editor, select the TilesGen game object and set the image filename in the inspector to your image.

Select the MainCamera game object and change the position x to 70, y to 70 and size to 100 as it'll be receiving a better result.

Click Play and you should be able to see your image as shown here:

Now, lets create a new script called Tile.

```

public enum Direction
{
    UP, DOWN, LEFT, RIGHT
}

49 references
public enum PosNegType
{
    POS,
    NEG,
    NONE,
}

```

```

public List<Vector2> CreateCurve(Direction dir, PosNegType type)
{
    int padding_x = mOffset.x;
    int padding_y = mOffset.y;
    int sw = titleSize;
    int sh = titleSize;

    List<Vector2> pts = new List<Vector2>(BezCurve);
    switch (dir)
    {
        case Direction.UP:
            if (type == PosNegType.POS)
            {
                TranslatePoints(pts, new Vector2(padding_x, sh + padding_y));
                return pts;
            }
            else if (type == PosNegType.NEG)
            {
                InvertY(pts);
                TranslatePoints(pts, new Vector2(padding_x, -padding_y));
                return pts;
            }
    }
}

```

```

    }
    break;
case Direction.DOWN:
    if (type == PosNegType.POS)
    {
        InvertY(pts);
        TranslatePoints(pts, new Vector2(padding_x, -padding_y));
    }
    else if (type == PosNegType.NEG)
    {
        TranslatePoints(pts, new Vector2(padding_x, sh + padding_y));
    }
    else
    {
        pts.Clear();
        for (int i = 0; i < 100; ++i)
        {
            pts.Add(new Vector2(i + padding_x, -padding_y));
        }
    }
    break;
case Direction.LEFT:
    if (type == PosNegType.POS)
    {
        InvertY(pts);
        SwapXY(pts);
        TranslatePoints(pts, new Vector2(-padding_x, -padding_y));
    }
    else if (type == PosNegType.NEG)
    {
        SwapXY(pts);
        TranslatePoints(pts, new Vector2(-padding_x, padding_y));
    }
    else
    {
        pts.Clear();
        for (int i = 0; i < 100; ++i)
        {
            pts.Add(new Vector2(-padding_x, i + padding_y));
        }
    }
    break;
}
return pts;
}

```

```

public void HideAllCurves()
{
    foreach(var item in mLineRenderers)
    {
        item.Value.gameObject.SetActive(false);
    }
}

2 references
public void DestroyAllCurves()
{
    foreach(var item in mLineRenderers)
    {
        GameObject.Destroy(item.Value.gameObject);
    }

    mLineRenderers.Clear();
}

```

```

private (Tile.PosNegType, UnityEngine.Color) GetRandomType()
{
    Tile.PosNegType type = Tile.PosNegType.POS;
    UnityEngine.Color color = UnityEngine.Color.blue;
    float rand = UnityEngine.Random.Range(0f, 1f);

    if(rand < 0.5f)
    {
        type = Tile.PosNegType.POS;
        color = UnityEngine.Color.blue;
    }
    else
    {
        type = Tile.PosNegType.NEG;
        color = UnityEngine.Color.red;
    }
    return (type, color);
}

```

```

// Update is called once per frame
☑ Unity Message | 0 references
void Update()
{
    if (Input.GetKeyDown(KeyCode.Space))
    {
        TestRandomCurves();
    }
}

```



```

void TestRandomCurves()
{
    if(mTile != null)
    {
        mTile.DestroyAllCurves();
        mTile = null;
    }

    mTile = new Tile(mTextureOriginal);
    mTile = tile;

    var type_color = GetRandomType();
    mTile.DrawCurve(Tile.Direction.UP, type_color.Item1, type_color.Item2);
    type_color = GetRandomType();
    mTile.DrawCurve(Tile.Direction.RIGHT, type_color.Item1, type_color.Item2);
    type_color = GetRandomType();
    mTile.DrawCurve(Tile.Direction.DOWN, type_color.Item1, type_color.Item2);
    type_color = GetRandomType();
    mTile.DrawCurve(Tile.Direction.LEFT, type_color.Item1, type_color.Item2);

    SpriteRenderer spriteRenderer = gameObject.GetComponent<SpriteRenderer>();
    spriteRenderer.sprite = mSprite;
}

```

```

public void DestroyAllCurves()
{
    foreach(var item in mLineRenderers)
    {
        GameObject.Destroy(item.Value.gameObject);
    }

    mLineRenderers.Clear();
}

```

Go to the Unity editor and select the TileGen game object and change the position z value to 1 as it ensures that the image is behind the curve in order of drawing

Click Play and view the possible configurations of Jigsaw tiles.

Now, we will continue the implantation in the Tile script as we already have a variable called mOriginalTexture. We will create a new texture called finalCut and alter the new texture.

```

public Texture2D finalCut { get; private set; }

```

Also, we'll add a new static readonly variable called TransparentColor and initiate it to zero for all components.

```

public static readonly Color TransparentColor = new Color(0.0f, 0.0f, 0.0f, 0.0f);

private PosNegType[] mCurveTypes = new PosNegType[4]
{
    PosNegType.NONE,
    PosNegType.NONE,
    PosNegType.NONE,
    PosNegType.NONE,
};

```

Let's now create a constructor for the Tile class as it initialises a Tile object with a given Texture2D as an input parameter.

First, we will calculate tileSizeWithPadding as twice the padding plus the tileSize as it activates the mOriginalTexture with the provided texture.

Then, it'll enable the Texture2D object named finalCut with dimensions tileSizeWithPadding by tileSizeWithPadding.

```

public Tile(Texture2D texture)
{
    mOriginalTexture = texture;
    int padding = mOffset.x;
    int tileSizeWithPadding = 2 * padding + tileSize;

    finalCut = new Texture2D(tileSizeWithPadding, tileSizeWithPadding, TextureFormat.ARGB32, false);

    // Initialize the finalCut texture to be fully transparent
    for (int y = 0; y < finalCut.height; y++)
    {
        for (int x = 0; x < finalCut.width; x++)
        {
            finalCut.SetPixel(x, y, TransparentColor);
        }
    }
}

```

Afterwards, we create a public function called Apply as it's the main method that cuts the image into a tile based on the curves and modifies the textures. There are two functions called FloodFillInit and FloodFill. Therefore, these two functions will make the necessary changes to the image finalCut.

```

public void Apply()
{
    FloodFillInit();
    FloodFill();
    finalCut.Apply();
}

```

Flood Fill Algorithm

A Flood Fill Algorithm that identifies and labels the connected component that a particular cell belongs to in a multidimensional array. For example, if a neighbouring pixel has the same colour as the selected one, it gets filled with a new colour. Then the process continues, spreading outward until it hits a boundary.

We will use the method named FloodFillInit to perform the Flood Fill Algorithm.

```
// A 2D boolean array that stores a mask indicating which pixels have been filled  
private bool[,] mVisited;
```

The other variable called mStack as it was used to store coordinates of pixels that need to be processed or visited during a flood fill operation. The stack data structure follows the Last in First Out principle which means that the last element added to the stack will be the first one to be removed. Therefore, in the flood fill the stack is used to keep track of the pixels that need to be filled with colour.

```
// A stack used for the flood fill of the texture algorithm  
private Stack<Vector2Int> mStack = new Stack<Vector2Int>();
```

Then add two indices' variables need to track the traversal of the 2D array.

```
public int xIndex = 0;  
public int yIndex = 0;
```

Initialising Flood Fill

This method calculates the size of the tile with padding included also it initialises the 2D Boolean array mVisited with dimensions corresponding to the tile's size which ensures each pixel's visitations can be tracked. All the elements of mVisited are initially set to false to indicate that no pixels have been visited yet.

```

void FloodFillInit()
{
    int padding = mOffset.x;
    int tileSizeWithPadding = 2 * padding + tileSize;

    mVisited = new bool[tileSizeWithPadding, tileSizeWithPadding];
    for (int i = 0; i < tileSizeWithPadding; ++i)
    {
        for (int j = 0; j < tileSizeWithPadding; ++j)
        {
            mVisited[i, j] = false;
        }
    }

    List<Vector2> pts = new List<Vector2>();
    for (int i = 0; i < mCurveTypes.Length; ++i)
    {
        pts.AddRange(CreateCurve((Direction)i, mCurveTypes[i]));
    }

    // Now we should have a closed curve
    for (int i = 0; i < pts.Count; ++i)
    {
        mVisited[(int)pts[i].x, (int)pts[i].y] = true;
    }

    // Start from the center of the tile
    Vector2Int start = new Vector2Int(tileSizeWithPadding / 2, tileSizeWithPadding / 2);

    mVisited[start.x, start.y] = true;
    mStack.Push(start);
}

```

Implement Flood Fill

Now, we'll implement the Flood Fill method as it initiates the flood fill operation for the tile as it calculates the width and height of the tile with padding and tileSize values.

Then, it processes pixels that are stored in the mStack until there are no more pixels left to process. For each pixel popped from the stack, as it fills the corresponding pixel in the finalCut texture using the Fill utility method we implemented earlier.

Next, we will add the four directions; right, left, up, down. For each direction, it checks whether if it will fall into bounds within the tile.

This process continues until all connected pixels within the tiles are filled to ensure the entire region enclosed by the curve is coloured.

```

void FloodFill()
{
    int padding = mOffset.x;
    int width_height = padding * 2 + tileSize;

    while(mStack.Count > 0)
    {
        Vector2Int v = mStack.Pop();

        int xx = v.x;
        int yy = v.y;

        Fill(v.x, v.y);

        int x = xx + 1;
        int y = yy;

        if(x < width_height)
        {
            Color c = finalCut.GetPixel(x, y);
            if (!mVisited[x, y])
            {
                mVisited[x, y] = true;
                mStack.Push(new Vector2Int(x, y));
            }
        }

        // check left
        x = xx - 1;
        y = yy;
        if(x > 0)
        {
            Color c = finalCut.GetPixel(x, y);
            if (!mVisited[x, y])
            {
                mVisited[x, y] = true;
                mStack.Push(new Vector2Int(x, y));
            }
        }

        // Check up
        x = xx;
        y = yy;

        if(y < width_height)
        {
            Color c = finalCut.GetPixel(x, y);
            if (!mVisited[x, y])
            {
                mVisited[x, y] = true;
                mStack.Push(new Vector2Int(x, y));
            }
        }

        // Check down
        x = xx;
        y = yy - 1;

        if(y < width_height)
        {
            Color c = finalCut.GetPixel(x, y);
            if (!mVisited[x, y])
            {
                mVisited[x, y] = true;
                mStack.Push(new Vector2Int(x, y));
            }
        }
    }
}

```

This method is the important part of cutting and filling the image for a jigsaw tile also its very error prone.

NOTE: Pay attention as it's hard to debug if you make any mistake coding this method.

Now, we will test the jigsaw tile creation based on the flood fill method. Go to the TileGen and add a new section in the existing Update method which is called TestFillFloodFill.

```

void Update()
{
    if (Input.GetKeyDown(KeyCode.Space))
    {
        TestRandomCurves();
    }
    else if (Input.GetKeyDown(KeyCode.C))
    {
        TestTileFloodFill();
    }

    void TestRandomCurves()
    {
        if(mTile != null)
        {
            mTile.DestroyAllCurves();
            mTile = null;
        }
    }
}

```

Testing Jigsaw Tile Creation

Next, we will implement the TestTileFloodFill method as it's designed to test the flood fill function of the tile component. As it checks if the mTile reference is not null; if so, it clears all curves drawn on the tile and sets the reference to null.

```
void TestRandomCurves()
{
    if(mTile != null)
    {
        mTile.DestroyAllCurves();
        mTile = null;
    }

    mTile = new Tile(mTextureOriginal);
    mTile = tile;

    var type_color = GetRandomType();
    mTile.DrawCurve(Tile.Direction.UP, type_color.Item1, type_color.Item2);
    type_color = GetRandomType();
    mTile.DrawCurve(Tile.Direction.RIGHT, type_color.Item1, type_color.Item2);
    type_color = GetRandomType();
    mTile.DrawCurve(Tile.Direction.DOWN, type_color.Item1, type_color.Item2);
    type_color = GetRandomType();
    mTile.DrawCurve(Tile.Direction.LEFT, type_color.Item1, type_color.Item2);

    SpriteRenderer spriteRenderer = gameObject.GetComponent<SpriteRenderer>();
    spriteRenderer.sprite = mSprite;
}
```

Next, go onto the Unity editor and press play and see the cutout jigsaw tile come to life. Keep pressing 'F' and see the carious jigsaw pieces.

Final Script:

TempleControlPoints:

```

using NUnit.Framework;
using System.Collections.Generic;
using UnityEngine;

© Unity Script | 1 reference
public class TemplebeizerCurve : MonoBehaviour
{
    public static readonly List<Vector2> templateControlPoints = new List<Vector2>
    {
        new Vector2(0, 0),
        new Vector2(35, 15),
        new Vector2(47, 13),
        new Vector2(45, 5),
        new Vector2(48, 0),
        new Vector2(25, -5),
        new Vector2(15, -18),
        new Vector2(36, -20),
        new Vector2(64, -20),
        new Vector2(85, -18),
        new Vector2(75, -5),
        new Vector2(52, 0),
        new Vector2(55, 5),
        new Vector2(53, 13),
        new Vector2(65, 15),
        new Vector2(100, 0)
    };

    public GameObject PointPrefab;

    LineRenderer[] mLineRenderers = null;

    List<GameObject> mPointGameObjects = new List<GameObject>();

    public float LineWidth;
    public float LineWidthBezier;
    public Color LineColor = new Color(0.5f, 0.5f, 0.5f, 0.8f);
    public Color LineColorBezier = new Color(1.0f, 0.0f, 0.0f, 0.8f);

    2 references
    private LineRenderer CreateLine()
    {
        GameObject obj = new GameObject();
        LineRenderer lr = obj.AddComponent<LineRenderer>();
        lr.material = new Material(Shader.Find("Sprites/Default"));
        lr.startColor = LineColor;
        lr.endColor = LineColor;
        lr.startWidth = LineWidth;
        lr.endWidth = LineWidth;
        return lr;
    }

    private void Start()
    {
        mLineRenderers = new LineRenderer[2];
        mLineRenderers[0] = CreateLine();
        mLineRenderers[1] = CreateLine();

        mLineRenderers[0].gameObject.name = "LineRenderer_obj_0";
        mLineRenderers[1].gameObject.name = "LineRenderer_obj_1";

        for (int i = 0; i < templateControlPoints.Count; i++)
        {
            GameObject obj = Instantiate(PointPrefab, templateControlPoints[i], Quaternion.identity);
            obj.name = "ControlPoint_" + i.ToString();
            mPointGameObjects.Add(obj);
        }
    }

    © Unity Message | 0 references
    private void Update()
    {
        LineRenderer lineRenderer = mLineRenderers[0];
        LineRenderer curveRenderer = mLineRenderers[1];

        List<Vector2> pts = new List<Vector2>();
        for (int i = 0; i < mPointGameObjects.Count; i++)
        {
            pts.Add(mPointGameObjects[i].transform.position);
        }

        lineRenderer.positionCount = pts.Count;
        for (int i = 0; i < pts.Count; i++)
        {
            lineRenderer.SetPosition(i, pts[i]);
        }

        List<Vector2> curve = BezierCurve.PointsList2(pts, 0.01f);
        curveRenderer.startColor = LineColorBezier;
        curveRenderer.endColor = LineColorBezier;
        curveRenderer.positionCount = curve.Count;
        curveRenderer.startWidth = LineWidthBezier;
        curveRenderer.endWidth = LineWidthBezier;

        for (int i = 0; i < curve.Count; i++)
        {
            curveRenderer.SetPosition(i, curve[i]);
        }
    }
}

```

Tile:

```
using System.Collections.Generic;
using UnityEditor;
using UnityEngine;
using UnityEngine.Rendering;
using UnityEngine.UIElements;

28 references
public class Tile
{
    23 references
    public enum Direction
    {
        UP, DOWN, LEFT, RIGHT
    }

    23 references
    public enum PosNegType
    {
        POS,
        NEG,
        NONE,
    }

    public Vector2Int mOffset = new Vector2Int(20, 20);

    public int tileSize = 100;

    private Dictionary<Direction, PosNegType> mLineRenderers
    = new Dictionary<Direction, PosNegType>(), LineRenderer>();

    public static List<Vector2> BezCurve =
        BezierCurve.PointsList2(TemplebeizerCurve.templateControlPoints, 0.001f);

    // The original texture from which the tile is created
    private Texture2D mOriginalTexture;

    10 references
    public Texture2D finalCut { get; private set; }

    public static readonly Color TransparentColor = new Color(0.0f, 0.0f, 0.0f, 0.0f);

    private PosNegType[] mCurveTypes = new PosNegType[4]
    {
        PosNegType.NONE,
        PosNegType.NONE,
        PosNegType.NONE,
        PosNegType.NONE,
    };

    // A 2D boolean array that stores a mask indicating which pixels have been filled

    private bool[,] mVisited;

    // A stack used for the flood fill of the texture algorithm
    private Stack<Vector2Int> mStack = new Stack<Vector2Int>();

    public void SetCurveType(Direction dir, PosNegType type)
    {
        mCurveTypes[(int)dir] = type;
    }

    0 references
    public PosNegType GetCurveType(Direction dir)
    {
        return mCurveTypes[(int)dir];
    }

    1 reference
    public Tile(Texture2D texture)
    {
        mOriginalTexture = texture;
        int padding = mOffset.x;
        int tileSizeWithPadding = 2 * padding + tileSize;

        finalCut = new Texture2D(tileSizeWithPadding, tileSizeWithPadding, TextureFormat.ARGB32, false);

        // Initialize the finalCut texture to be fully transparent
        for (int y = 0; y < finalCut.height; y++)
        {
            for (int x = 0; x < finalCut.width; x++)
            {
                finalCut.SetPixel(x, y, TransparentColor);
            }
        }
    }

    0 references
    public void Apply()
    {
        FloodFillInit();
        FloodFill();
        finalCut.Apply();
    }

    1 reference
    void FloodFillInit()
    {

```



```

public void Apply()
{
    FloodFillInit();
    FloodFill();
    finalCut.Apply();
}

1 reference
void FloodFillInit()
{
    int padding = mOffset.x;
    int titleSizeWithPadding = 2 * padding + titleSize;

    mVisited = new bool[titleSizeWithPadding, titleSizeWithPadding];
    for (int i = 0; i < titleSizeWithPadding; ++i)
    {
        for (int j = 0; j < titleSizeWithPadding; ++j)
        {
            mVisited[i, j] = false;
        }
    }

    List<Vector2> pts = new List<Vector2>();
    for (int i = 0; i < mCurveTypes.Length; ++i)
    {
        pts.AddRange(CreateCurve((Direction)i, mCurveTypes[i]));
    }

    // Now we should have a closed curve
    for (int i = 0; i < pts.Count; ++i)
    {
        mVisited[(int)pts[i].x, (int)pts[i].y] = true;
    }

    // Start from the center of the tile
    Vector2Int start = new Vector2Int(titleSizeWithPadding / 2, titleSizeWithPadding / 2);

    mVisited[start.x, start.y] = true;
    mStack.Push(start);
}

1 reference
void FloodFill()
{
    int padding = mOffset.x;
    int width_height = padding * 2 + titleSize;

    while(mStack.Count > 0)
    {
        Vector2Int v = mStack.Pop();

        int xx = v.x;

```

```

void FloodFill()
{
    int padding = mOffset.x;
    int width_height = padding * 2 + titleSize;

    while(mStack.Count > 0)
    {
        Vector2Int v = mStack.Pop();

        int xx = v.x;
        int yy = v.y;

        Fill(v.x, v.y);

        int x = xx + 1;
        int y = yy;

        if(x < width_height)
        {
            Color c = finalCut.GetPixel(x, y);
            if (!mVisited[x, y])
            {
                mVisited[x, y] = true;
                mStack.Push(new Vector2Int(x, y));
            }
        }

        // check left
        x = xx - 1;
        y = yy;
        if(x > 0)
        {
            Color c = finalCut.GetPixel(x, y);
            if (!mVisited[x, y])
            {
                mVisited[x, y] = true;
                mStack.Push(new Vector2Int(x, y));
            }
        }

        // Check up
        x = xx;
        y = yy;

        if( y < width_height)
        {
            Color c = finalCut.GetPixel(x, y);
            if (!mVisited[x, y])
            {
                mVisited[x, y] = true;
                mStack.Push(new Vector2Int(x, y));
            }
        }
    }
}

```

```

        // Check down
        x = xx;
        y = yy - 1;

        if (y < width_height)
        {
            Color c = finalCut.GetPixel(x, y);
            if (!mVisited[x, y])
            {
                mVisited[x, y] = true;
                mStack.Push(new Vector2Int(x, y));
            }
        }
    }
}

// Add this method to the Tile class to resolve CS0103: The name 'Fill' does not exist in the current context
1 reference
private void Fill(int x, int y)
{
    // Example implementation: fill the pixel at (x, y) with a solid color (e.g., white)
    // You may want to adjust this logic to fit your application's requirements
    finalCut.SetPixel(x, y, Color.white);
}

1 reference
public static LineRenderer CreateLineRenderer(UnityEngine.Color color, float lineWidth = 1.0f)
{
    GameObject obj = new GameObject();
    LineRenderer lr = obj.AddComponent<LineRenderer>();

    lr.startColor = color;
    lr.endColor = color;
    lr.startWidth = lineWidth;
    lr.endWidth = lineWidth;
    lr.material = new Material(Shader.Find("Sprites/Default"));
    return lr;
}

8 references
public static void TranslatePoints(List<Vector2> iList, Vector2 offset)
{
    for(int i = 0; i < iList.Count; i++)
    {
        iList[i] += offset;
    }
}

0 references
public static void Invert(List<Vector2> iList)
{
    for(int i = 0; i < iList.Count; i++)
    {
        iList[i] = -iList[i];
    }
}

```

```

public static void Invert(List<Vector2> ilist)
{
    for(int i = 0; i < ilist.Count; i++)
    {
        ilist[i] = -ilist[i];
    }
}

4 references
public static void SwapXY(List<Vector2> ilist)
{
    for(int i = 0; i < ilist.Count; ++i)
    {
        ilist[i] = new Vector2(ilist[i].y, ilist[i].x);
    }
}

4 references
public static void InvertY(List<Vector2> ilist)
{
    for (int i = 0; i < ilist.Count; i++)
    {
        ilist[i] = new Vector2(ilist[i].x, -ilist[i].y);
    }
}

2 references
public List<Vector2> CreateCurve(Direction dir, PosNegType type)
{
    int padding_x = mOffset.x;
    int padding_y = mOffset.y;
    int sw = titleSize;
    int sh = titleSize;

    List<Vector2> pts = new List<Vector2>(BezCurve);
    switch (dir)
    {
        case Direction.UP:
            if (type == PosNegType.POS)
            {
                TranslatePoints(pts, new Vector2(padding_x, sh + padding_y));
                return pts;
            }
            else if (type == PosNegType.NEG)
            {
                InvertY(pts);
                TranslatePoints(pts, new Vector2(padding_x, -padding_y));
                return pts;
            }
            else
            {
                pts.Clear();
                for (int i = 0; i < 100; ++i)
                {

```

```

public List<Vector2> CreateCurve(Direction dir, PosNegType type)
{
    int padding_x = mOffset.x;
    int padding_y = mOffset.y;
    int sw = titleSize;
    int sh = titleSize;

    List<Vector2> pts = new List<Vector2>(BezCurve);
    switch (dir)
    {
        case Direction.UP:
            if (type == PosNegType.POS)
            {
                TranslatePoints(pts, new Vector2(padding_x, sh + padding_y));
                return pts;
            }
            else if (type == PosNegType.NEG)
            {
                InvertY(pts);
                TranslatePoints(pts, new Vector2(padding_x, -padding_y));
                return pts;
            }
            else
            {
                pts.Clear();
                for (int i = 0; i < 100; ++i)
                {
                    pts.Add(new Vector2(i + padding_x, padding_y + sh));
                }
            }
            break;
        case Direction.RIGHT:
            if (type == PosNegType.POS)
            {
                SwapXY(pts);
                TranslatePoints(pts, new Vector2(padding_x + sw, padding_y));
            }
            else if (type == PosNegType.NEG)
            {
                InvertY(pts);
                SwapXY(pts);
                TranslatePoints(pts, new Vector2(-padding_x, padding_y));
            }
            else
            {
                pts.Clear();
                for (int i = 0; i < 100; ++i)
                {
                    pts.Add(new Vector2(padding_x + sw, i + padding_y));
                }
            }
            break;
    }
}

```

```

    }
    break;
case Direction.DOWN:
    if (type == PosNegType.POS)
    {
        InvertY(pts);
        TranslatePoints(pts, new Vector2(padding_x, -padding_y));
    }
    else if (type == PosNegType.NEG)
    {
        TranslatePoints(pts, new Vector2(padding_x, sh + padding_y));
    }
    else
    {
        pts.Clear();
        for (int i = 0; i < 100; ++i)
        {
            pts.Add(new Vector2(i + padding_x, -padding_y));
        }
    }
    break;
case Direction.LEFT:
    if (type == PosNegType.POS)
    {
        InvertY(pts);
        SwapXY(pts);
        TranslatePoints(pts, new Vector2(-padding_x, -padding_y));
    }
    else if (type == PosNegType.NEG)
    {
        SwapXY(pts);
        TranslatePoints(pts, new Vector2(-padding_x, padding_y));
    }
    else
    {
        pts.Clear();
        for (int i = 0; i < 100; ++i)
        {
            pts.Add(new Vector2(-padding_x, i + padding_y));
        }
    }
    break;
}
return pts;

```

```

public void DrawCurve(Direction dir, PosNegType type, UnityEngine.Color color)
{
    if(!mLineRenderers.ContainsKey((dir,type)))
    {
        mLineRenderers.Add((dir, type), CreateLineRenderer(color));
    }

    LineRenderer lr = mLineRenderers[(dir, type)];
    lr.startColor = color;
    lr.endColor = color;
    lr.gameObject.name = "LineRenderer_" + dir.ToString() + "_" + type.ToString();
    List<Vector2> pts = CreateCurve(dir, type);

    lr.positionCount = pts.Count;
    for(int i = 0; i < pts.Count; ++i)
    {
        lr.SetPosition(i, pts[i]);
    }
}

1 reference
public void HideAllCurves()
{
    foreach(var item in mLineRenderers)
    {
        item.Value.gameObject.SetActive(false);
    }
}

0 references
public void DestroyAllCurves()
{
    foreach(var item in mLineRenderers)
    {
        GameObject.Destroy(item.Value.gameObject);
    }

    mLineRenderers.Clear();
}

```

TileGen:

```

using UnityEngine;

@ Unity Script | 0 references
public class TileGen : MonoBehaviour
{
    // Start is called once before the first execution of Update after the MonoBehaviour is created
    public string imageFilename;
    private Texture2D mTextureOriginal;
    private Tile mTile;

    @ Unity Message | 0 references
    void Start()
    {
        CreateBaseTexture();

        mTile = new Tile(mTextureOriginal);

        mTile.DrawCurve(Tile.Direction.UP, Tile.PosNegType.POS, UnityEngine.Color.blue);
        mTile.DrawCurve(Tile.Direction.RIGHT, Tile.PosNegType.POS, UnityEngine.Color.blue);
        mTile.DrawCurve(Tile.Direction.DOWN, Tile.PosNegType.POS, UnityEngine.Color.blue);
        mTile.DrawCurve(Tile.Direction.UP, Tile.PosNegType.POS, UnityEngine.Color.blue);

        mTile.DrawCurve(Tile.Direction.UP, Tile.PosNegType.NEG, UnityEngine.Color.blue);
        mTile.DrawCurve(Tile.Direction.RIGHT, Tile.PosNegType.NEG, UnityEngine.Color.blue);
        mTile.DrawCurve(Tile.Direction.DOWN, Tile.PosNegType.NEG, UnityEngine.Color.blue);
        mTile.DrawCurve(Tile.Direction.UP, Tile.PosNegType.NEG, UnityEngine.Color.blue);
    }

    1 reference
    void CreateBaseTexture()
    {
        mTextureOriginal = SpriteUtils.LoadTexture(imageFilename);
        if(!mTextureOriginal.isReadable)
        {
            Debug.Log("Texture load is unreadable ");
            return;
        }

        SpriteRenderer spriteRenderer = gameObject.GetComponent<SpriteRenderer>();
        // Replace the call to the non-existent CreateSpriteTexture2D with the correct method CreateSpriteFreeTexture2D
        spriteRenderer.sprite = SpriteUtils.CreateSpriteFreeTexture2D(
            mTextureOriginal,
            0,
            0,
            mTextureOriginal.width,
            mTextureOriginal.height);
    }

    private (Tile.PosNegType, UnityEngine.Color) GetRandomType()
    {
        Tile.PosNegType type = Tile.PosNegType.POS;
        UnityEngine.Color color = UnityEngine.Color.blue;
        float rand = UnityEngine.Random.Range(0f, 1f);

        if(rand < 0.5f)
        {
            type = Tile.PosNegType.POS;
            color = UnityEngine.Color.blue;
        }
        else
        {
            type = Tile.PosNegType.NEG;
            color = UnityEngine.Color.red;
        }
        return (type, color);
    }

    // Update is called once per frame
    @ Unity Message | 0 references
    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Space))
        {
            mTile.HideAllCurves();

            var type_color = GetRandomType();
            mTile.DrawCurve(Tile.Direction.UP, type_color.Item1, type_color.Item2);
            type_color = GetRandomType();
            mTile.DrawCurve(Tile.Direction.RIGHT, type_color.Item1, type_color.Item2);
            type_color = GetRandomType();
            mTile.DrawCurve(Tile.Direction.DOWN, type_color.Item1, type_color.Item2);
            type_color = GetRandomType();
            mTile.DrawCurve(Tile.Direction.LEFT, type_color.Item1, type_color.Item2);
        }
    }
}

```

SpriteUtils:


```
using UnityEngine;

2 references
public class SpriteUtils
{
    1 reference
    public static Sprite CreateSpriteFreeTexture2D(
        Texture2D spriteTexture,
        int x,
        int y,
        int w,
        int h,
        float pixelPerUnit = 1.0f,
        SpriteMeshType spriteType = SpriteMeshType.Tight)
    {
        Sprite newSprite = Sprite.Create(
            spriteTexture,
            new Rect(x, y, w, h),
            new Vector2(0.5f, 0.5f),
            pixelPerUnit,
            0,
            spriteType);
        return newSprite;
    }

    1 reference
    public static Texture2D LoadTexture(string resourcePath)
    {
        Texture2D tex = Resources.Load<Texture2D>(resourcePath);
        return tex;
    }
}
```